

#interfaceType and #interface: how TestController decides which of its features can see a device

A driver declares `#interfaceType PS`. What does that do? Does it give the driver the power-supply interface, does TestController check that the driver lives up to the claim, and what decides whether the device turns up in the Power supply test, the Battery test, or a panel dropdown?

What this research found, in five points:

1. `#interface` and `#interfaceType` are two independent lists. Neither one populates or validates the other.
2. `#interfaceType` is a classification tag. Declaring `PS` injects no functions, and TestController never checks that a `PS` device can do what a power supply does.
3. `#interface` is a name-to-action map. Each entry is one callable function, and what exists is what the driver author wrote down.
4. Every feature that offers a list of devices applies its own fixed pair: one type pattern, plus a list of functions that must all exist. Both have to pass.
5. Failing either half removes the device from that feature's list silently. There is no error, no log line, and nothing in the UI to say a device was considered and dropped.

The rest of this note is the detail behind those points, ending with a table of every such pair in the program.

Describes TestController 3.49. Revised 21 August 2026. Details of this kind change between releases; anything here that a later version contradicts is out of date rather than disputed.

Authority. This is a working note, assembled from the published documentation, from the author's public posts in the EEVblog thread, and from watching TestController behave while writing drivers for it. It is not documentation. The author has read a draft and corrected it, and those corrections are incorporated here — but he declined to comment on which popup or panel uses which function, so section 3 and Appendix A, the broadest claims in the note, are unreviewed. HKJ is the author of TestController and the only authority on what it is supposed to do; where this note and he disagree, he is right and this note is a bug report. Everything attributed to him below is a quotation from a public source, listed under Sources at the end.

1. The two directives

#interface : a map of callable names to actions

```
#interface setVoltage VOLT (value)
#interface getVoltage VOLT?
#interface readVoltage 0
```

Each line binds a name to an action, and the name's prefix decides how the action is interpreted:

Prefix	Rest of the line is	What happens when called
set...	a device command, with <code>(value)</code> where the argument goes	the command is sent
get...	a device query	the query is sent and the reply returned, a fresh round trip

Prefix	Rest of the line is	What happens when called
read...	column indices, not a command — one index per channel, space separated	a sample of every loaded device is requested unless one landed very recently, then that column's value is returned
name..., unit...	column indices	the column's name or unit, resolved the same way as read..., with no sampling

The `read...` prefix behaves differently from the way its name reads, both in the argument it takes and in what calling it costs.

`readVoltage 0` sends no command of its own. The `0` is a column of the value set, the same columns `#askValues` fills and the log records, so the answer is a measured value rather than a readback. If a mode change has rewritten the column layout, index 0 now means something else. `getVoltage` and `readVoltage` on the same driver answer two different questions: what was asked for, and what was last measured. The author's guidance for setpoints is to use the `get...` half: "You do not need read... functions for setpoints, there the get... functions work" ([reply #7270](#)). On a multi-channel driver the entry carries one index per channel, space separated, and the channel argument picks which one is used, which he set out in 2024 as "for multichannel devices multiple indexes must be listed, one for each channel" ([reply #3319](#)). An entry may also carry a `:decodemath:` line, whose expression post-processes the column value with `value` and `channel` in scope.

Calling a `read...` function also asks for a fresh sample first, of every loaded device rather than only the one named. That request is suppressed if a sample landed recently, where "recently" is derived from the measured command time and clamped to a window between 100 ms and 1.5 s. Back-to-back reads inside that window are cheap, and reads spaced further apart each drive a full acquisition cycle across the bench. A `read...` in a tight script loop therefore costs more than it appears to, and the cost is not confined to the device being read.

Two entries can appear without being written. With two or more `#cmdMode` blocks TestController adds `setMode` itself, and adds `getMode` as well if `#askMode` is present. Everything else in the map is what the author typed.

`set...` and `get...` work as a pair, which is why the function lists in section 3 almost always ask for both halves. `getX` returns the setting that `setX` wrote, so it answers what the setting is in the device right now, and it lets a caller read a value back rather than assume the write landed. Reading it as a verification mechanism, and reading a feature that asks for `setVoltage` and `getVoltage` together as asking for a parameter it can check, is this note's inference from the pattern rather than a documented rule.

#interfaceType : a list of classification tags

```
#interfaceType PS
#interfaceType ps ps:2 ps:3
#interfaceType humidity thermometer
```

A list of tags, separated by commas or whitespace, each optionally carrying a `:n` channel number. It injects nothing, requires nothing, and is never compared against the interface map.

The tags are normalised before anything is matched against them, and the normalised form is what the features compare against:

You write	Stored as	Rule
ps, Ps, PS	PS	one of the nine names TC knows is upper-cased
acps, arb, dmm, bmm, lcr, pc, sa, pwm	upper case	same list
load, LOAD, Load	Load	anything else gets first letter up, rest down

You write	Stored as	Rule
readout, thermometer, relay, charger	Readout, Thermometer, ...	same
ainput, aoutput	AInput, AOutput	special-cased pair
VBG, VHG, VLG, VMSG, VPG, VQG, VRG, VSG, VUG	Virtual	the nine virtual-device tags all collapse to one
ps:1, ps:0	PS	channel 1 is the default, so it is spelled by omission
ps:2	PS:2	any other number is kept

Because of this normalisation the case typed in the driver does not matter. It is the canonical spelling in the middle column that every feature compares against, which is why the tables later in this note use that form.

How the tag list is split and normalised has moved more recently than the matching it feeds. In March 2026 a meter declared as `#interfaceType DMM, BMM` was not recognised as a DMM, and the advice at the time was to "get rid of the comma, only valid delimiter is space in this case" ([reply #5924](#)). In 3.49 the separators are comma, semicolon, space and line break, and that same line is accepted. The two are not in conflict; they are different releases. A tab is not among the separators in 3.49, so two tags separated by one are read as a single tag, which then matches nothing.

The requirement pairs and the patterns in section 3 are the older and more durable half of this note. This table is the half most likely to differ on another version, in either direction.

The channel numbers are where `#interfaceType` stops being purely descriptive. The highest `:n` in the list is the device's channel count, and several features use it to offer `HANDLE:2`, `HANDLE:3` and so on as separate entries. Declaring `ps ps:2 ps:3` does not create three sets of `#interface` definitions; it tells TestController that the single set should be offered three times with a channel argument. The author described the same line for a four-relay board as declaring "that the interface has 4 independent devices that is accessed by adding :2, :3 or :4 to the handle" ([reply #5870](#)), and put the bare tag and `:1` together as "OSC and OSC:1 has the same meaning" ([reply #1223](#)).

They never meet

Declaring a type adds no interface entry, and declaring interface entries adds no type. A driver can have a complete power-supply interface and no type at all, or `#interfaceType PS` and an empty interface map. TestController accepts both without comment.

HKJ says as much on the [interface-definitions page](#):

```
#interfaceType is not mandatory for a driver to implement, but is useful for classifying the driver, **it can be added without any #interface definitions.**
```

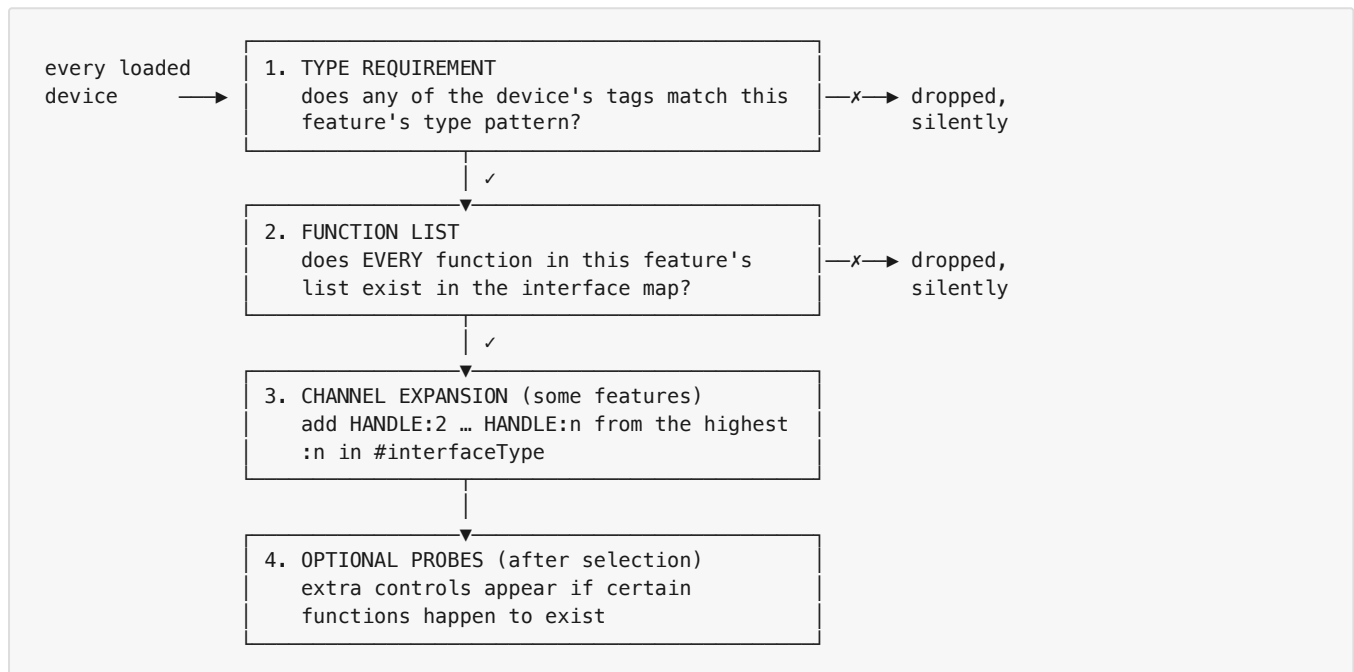
2. How they combine when a TC feature is looking for available assets

Every place in TestController that presents a list of devices, a popup's equipment search or a panel's device dropdown, narrows the loaded devices with two requirements of its own: a type pattern, and a list of functions that must all be present. Both are fixed in the feature. The author has stated one such pair in public in exactly

that shape: for a meter to be considered by the Battery test, "the meter must be type DMM and support readValue()" ([reply #5959](#)).

Two requirements, not one mechanism. An earlier draft of this note called the arrangement a "double gate". That overstates it, because it implies one mechanism applied uniformly, and the author describes something looser and decided feature by feature: naming conventions in the `set...` namespace are what "makes it possible to automatic use them in some popups", TestController "has no idea what the different number controls do, that is the reason it exposes all of them", and Steps "do not check device type" at all ([reply #7270](#)). Section 3 bears that out: the pairs differ from feature to feature, several features require no particular type, two require no functions at all, and Steps applies neither requirement. What follows is therefore the shape one feature's question takes, enumerated feature by feature, and not a single check the program runs everywhere.

The shape of that question, for a feature that uses both requirements:



Properties of the arrangement, as observed:

- The type requirement is applied first. The function list is only consulted for devices that already matched the type, so a device with the right functions and the wrong type is never examined.
- Every name in the function list must be present. The list is split on spaces, commas and semicolons, and the first missing name disqualifies the device. Seven of eight fails the same way zero of eight does.
- A device with no `#interfaceType` at all is invisible everywhere, including to features whose type pattern is `\*`. Such a driver is not treated as one with an empty type list; it is passed over before the question is asked. `\*` means any type, rather than no type required. This is the behaviour most likely to catch a driver author out, and it is stated in none of the three documentation pages.
- An empty function list means type only. Two features use this: the Remote readout popup and the DevicePopups panel. For them the tag alone is sufficient.
- A function list of exactly `\*` behaves the same as an empty list.
- Nothing is reported: no dialog, no status line, no entry in the log. From the outside, "this driver was never considered" and "this driver was considered and rejected for one missing function" look identical, as an absent row in a dropdown. Asked for a validator that would flag a definition missing its minimum functions, the author answered "I do not really like to hassle people about it, for me it is fine if a definition is missing

some element, I just hope somebody else will add it" ([reply #6020](#)), so the quiet appears to be a choice rather than an oversight.

How a type is matched

The comparison is a whole-tag match rather than a substring search. A feature that wants `PS` is satisfied by the tag `PS` and not by `PSU` or `MYPs`. Wildcards are available to a feature that wants a family rather than one name, and several use them to accept `PS` together with `PS:2`, `PS:3` and so on.

Most patterns accept channel-suffixed tags. Four do not, and those four are set out in Appendix A.

3. Every feature's requirements in TestController 3.49

This is the whole set. "Type" is what the pattern admits after normalisation; the patterns themselves are in Appendix A. "Ch" marks the features that expand multi-channel devices into `HANDLE:2`, `HANDLE:3`, ...

Popups

Feature	Slot	Type admitted	All of these #interface must exist	Ch
Power supply test	load	Load, Load:n	setCurrent getVoltage readVoltage readCurrent setOn	—
	source, 1st choice	ACPS, ACPS:n	setVoltage getVoltage setCurrent getVoltage readCurrent readVoltage readPower setOn	—
	source, 2nd choice	PS, PS:n	setVoltage getVoltage setCurrent getVoltage readCurrent readVoltage setOn	—
	source, 3rd choice	Power, Power:n	readCurrent readVoltage readPower	—
	temperature	Thermometer, Thermometer:n	readTemperature	✓
	meters	DMM (bare tag matches; see Appendix A)	readValue	—
Battery test	load	Load, Load:n	setCurrent getVoltage readVoltage readCurrent setOn	—
	supply	PS, PS:n	setVoltage getVoltage setCurrent getVoltage readCurrent readVoltage setOn	—
	relay	Relay, Relay:n	setRelays	—
	temperature	Thermometer, Thermometer:n	readTemperature	—
	meters	DMM (bare tag matches; see Appendix A)	readValue	—
MPPT	load	Load (bare tag matches; see Appendix A)	setCurrent readVoltage readCurrent	✓
Log Event	device	any type	setOn	✓

Feature	Slot	Type admitted	All of these #interface must exist	Ch
Log Event Define Commands	device	any type	setOn	✓
Remote readout	device	Readout , Readout:n	(none — type alone)	✓

One detail of the Power supply test's source list: the three source patterns are tried in order, and the first one that yields anything wins and fixes the test's mode as AC source, DC supply or wattmeter. A power meter is only offered when no ACPS and no PS qualified. The popup then takes the first device from each list without asking. There is no source or load dropdown, so with two qualifying supplies loaded you get whichever TestController enumerated first.

Panels

Panel	Type admitted	All of these must exist	Ch
OnOff, MultiOnOff	any type	setOn	✓
SetDualParamsWithOff, SetTripleParamsWithOff	any type	setOn	✓
VoltCurrentPowerReadout	any type	readVoltage readCurrent	✓
DevicePopups	any type	(none — type alone)	—
EfficiencyReadout — meter slot	any type	readValue	✓
EfficiencyReadout — main slot	any type	readCurrent	✓
EfficiencyReadout — aux slot	any type	readVoltage	✓
PSLoadReadout	PS , Load , Charger , and :n of each	readVoltage readCurrent	✓
MultiPSLoadReadout	PS , Load , and :n of each	readVoltage readCurrent	✓
SetPSVoltCurrentWithOff	PS , ACPS (bare tags match; see Appendix A)	setOn setVoltage setCurrent	✓
SetDualPSVoltCurrentWithOff	as above, two independent dropdowns	setOn setVoltage setCurrent	✓
SetMultiLoadsWithOff	Load (bare tag matches; see Appendix A)	setOn setCurrent	✓

Functions checked after a device is selected

Not every function a feature can use is in its required list. Some are optional, and are probed after a device has already been selected: if the function is there the feature offers an extra control, and if it is not the feature carries on without it. Absence from a required list is what makes them safe to leave out, since an optional function cannot cost a device its place in a dropdown.

Function	Probed by	What it buys	Why it is not required
setRemoteSense	Power supply test (on the load <i>and</i> on whichever source won), Battery test (on the load and the supply)	a remote-sense control for that device	optional on any supply or load
setFrequency + getFrequency	Power supply test, on an AC source	the frequency control	optional even for an AC source; both halves must exist

Function	Probed by	What it buys	Why it is not required
<code>readPower</code>	Power supply test, on the DC-supply path; MPPT; the readout panels	a power reading instead of a computed one	required only for AC, where it is in the required list, and optional for DC

The distinction matters when deciding what to implement. A function in a required list is all-or-nothing: leave it out and the device vanishes from that feature with no explanation. A function in the table above is purely additive.

This list came out of working through the program rather than out of the documentation, and it is not the output of an exhaustive search. Read it as "at least these" rather than "only these".

Steps

Steps applies no type requirement at all: "TC do not check device type when scanning function names for Steps" ([reply #7270](#)). It walks each loaded device's interface map and offers what it recognises:

Step kind	Offered for	Type required
on/off action	<code>setOn</code> , <code>setRelay</code> , <code>setOutput</code> , <code>setRemoteSense</code> — whichever exist, once per channel	none
string action	<code>setReadoutString</code> , if it exists	none
parameter sweep	every <code>#cmdSetup</code> number control, named <code>handle.page.label</code>	none — and no <code>#interface</code> involvement either

This is why a driver can appear in Steps but not in the Remote readout popup even though both involve `setReadoutString`. Steps only asks whether the function exists, while Remote readout asks for `#interfaceType` `Readout` first and never looks at the function.

Script surface

How	Type required	Functions required
<code>getDevice("PS")</code>	exact tag match, case-insensitive, <code>:n</code> aware; <code>*</code> returns all matches, a <code>+</code> prefix skips to the next match	none — the returned handle may not support anything
<code>#popupdevice</code>	whatever the script author passes	whatever the script author passes — the same two requirements, handed to the script
<code>#SCRIPTINTERFACE</code> type function ...	one type	all listed functions — controls whether a script-library entry is visible
calling <code>setVoltage("PS1", 5)</code> directly	none	resolved at call time; a missing name raises an unknown-function error naming the device

`#SCRIPTINTERFACE` hands the same two-part question to script authors in the same shape: one type, then N functions. It is the clearest statement in the documentation that this is the intended pattern.

4. The inverse view: what each tag buys, and what it costs

The same data turned around. If you declare this tag, these are the features that become reachable, and this is the function set needed for all of them.

PS

Function set	Unlocks
setVoltage getVoltage setCurrent getCurrent readVoltage readCurrent setOn	Power supply test (as source), Battery test (as supply)
setOn setVoltage setCurrent	SetPSVoltCurrentWithOff, SetDualPSVoltCurrentWithOff
readVoltage readCurrent	PSLoadReadout, MultiPSLoadReadout, VoltCurrentPowerReadout
setOn	OnOff, MultiOnOff, SetDual/TripleParamsWithOff, Log Event, Log Event Define Commands

The union is seven functions: setVoltage getVoltage setCurrent getCurrent readVoltage readCurrent setOn . That is the complete cost of full built-in support for a DC supply. getOn appears in the [Primary list](#) but no feature requires it; leaving it out costs nothing in 3.49, and implementing it hedges against that changing.

ACPS

Function set	Unlocks
setVoltage getVoltage setCurrent getCurrent readVoltage readCurrent readPower setOn	Power supply test (as AC source)
setOn setVoltage setCurrent	SetPSVoltCurrentWithOff, SetDualPSVoltCurrentWithOff
setOn	the any-type on/off panels and Log Event

ACPS is not in the PSLoadReadout or MultiPSLoadReadout patterns, so an AC supply does not appear in those readout panels however it is defined. Declaring acps ps gets it into both sets of features, at the cost of claiming to be two things.

Load

Function set	Unlocks
setCurrent getCurrent readVoltage readCurrent setOn	Power supply test (as load), Battery test (as load)
setCurrent readVoltage readCurrent	MPPT — the pattern matches the bare tag only, but the popup expands channels itself
setOn setCurrent	SetMultiLoadsWithOff — matched on the bare tag, and the panel then expands channels itself
readVoltage readCurrent	PSLoadReadout, MultiPSLoadReadout, VoltCurrentPowerReadout

The union is five functions: setCurrent getCurrent readVoltage readCurrent setOn .

Power , DMM , Thermometer , Relay , Readout , Charger

Tag	Functions needed	Unlocks	Caveat
Power	readCurrent readVoltage readPower	Power supply test, as wattmeter source	only when no ACPS and no PS qualified
DMM	readValue	Power supply test and Battery test meter lists	the device is admitted, but only its first channel is used — these lists do not expand channels
Thermometer	readTemperature	Power supply test and Battery test temperature lists	PS test expands channels, Battery test does not
Relay	setRelays	Battery test relay list	plural — setRelay alone is not enough
Readout	(none)	Remote readout popup	the popup then shows only the setReadout... variants that exist
Charger	readVoltage readCurrent	PSLoadReadout only	required by no other feature

Any tag at all

Functions	Unlocks
setOn	OnOff, MultiOnOff, SetDualParamsWithOff, SetTripleParamsWithOff, Log Event, Log Event Define Commands
readVoltage readCurrent	VoltCurrentPowerReadout
readValue	EfficiencyReadout meter slot
readCurrent or readVoltage	EfficiencyReadout main / aux slots
(none)	DevicePopups panel

No tag

Nothing. Not one of the features above, including the ones whose pattern is *. If a driver is missing from every dropdown, this is the first thing to check.

Tags with no built-in consumer in 3.49

ARB , BMM , LCR , PC , SA , PWM , Virtual , AInput , AOutput , Humidity , Input , Output , Calibrator .

These are real, documented, correctly normalised tags, and no built-in feature requires any of them. They serve `getDevice()` in scripts, `#SCRIPTINTERFACE` script visibility, and classification. Declaring one is not useless, but it is not a route to a popup either.

5. Traps

A driver with no tag

Covered above, and repeated here because the symptom is so unhelpful: a driver with a complete interface map and no `#interfaceType` line appears in nothing.

Misspelled tags are silently valid

Spelling is not normalised, only casing is. Unknown tags are accepted and capitalised, so a typo becomes a valid tag that no pattern matches, and nothing anywhere says so.

As of the 3.49 documentation this was a live hazard rather than a theoretical one. The [interface-definitions page](#) contained two misspellings in its own examples, `#interfaceType thermomenter` in the Thermometer section and again under Humidity, plus `#interfaceType humidity temperature` in the general section, where the tag the features look for is `thermometer`. Copying any of those three lines produced a device that never appeared in the Power supply test or Battery test temperature lists, with no indication why. That page may have been revised since. The underlying trap, that a misspelled tag is silently valid, is not a documentation bug and remains, and it surfaces in the thread from time to time as a definition that looks right, a device that is "NOT recognized", and nothing on screen to say which half failed ([reply #5924](#)).

What the channel number means at call time

Channel numbering starts at 1, and channel 1 is spelled by omission. A handle with no suffix is channel 1, not a channel-less device: `ScriptInterface` initialises the channel to `1` and overwrites it only when the handle carries a suffix, so `(channel)` in an `#interface` definition substitutes `1` for a bare handle. That is why the bare tag and `:1` "has the same meaning" ([reply #1223](#)) — they mean the same thing, so they normalise to the same thing. `:0` takes a different route to the same place: the index path subtracts one and clamps a negative result, so `:0` lands on the first channel rather than failing.

The suffix does double duty, and which duty it is doing is settled by the function's name rather than by the number. Channel-specific functions use it; global ones receive it and are expected to discard it. The clearest pair is `relay` against `relays`. `setRelay` and `getRelay` act on one relay and need the number; `setRelays` and `getRelays` take a bitmask for the whole device and each, in the author's words, "ignores the multichannel aspect of a relay definition" ([Interface definitions](#)). `readInput` and `readInputs` split the same way, and `setAllOff()` takes no parameter at all. The call may still pass a channel to any of them; the definition is what decides whether it means anything.

Nothing enforces this. `TestController` substitutes `(channel)` and exposes `channel` as a script variable in every case, so a global definition complies simply by not referring to either. It is the same kind of convention as the `set...` / `read...` / `get...` namespaces — carried in the name, honoured by the author, unchecked by the program.

Patterns that omit the channel clause

Four patterns omit it (Appendix A), and this matters less than it appears to. The type requirement is satisfied by any one of a device's tags, so a device declaring `dmm dmm:2` is admitted on its bare `dmm`. The missing clause only means the suffixed tag would not have matched on its own, which never arises while the bare tag is present. The omission bites exactly one case: a driver that declares a suffixed tag and no bare tag, such as `#interfaceType dmm:2` alone. Declaring the bare tag alongside the numbered ones keeps the question from arising.

Matching a device, and using its channels

These are separate decisions, and the second is where multi-channel devices lose something. A feature either expands a device into `HANDLE:2`, `HANDLE:3`, ... or takes the first channel and ignores the rest; the "Ch" column in section 3 says which. The Battery test expands nothing at all, so every one of its slots takes the first channel. The Power supply test expands only its temperature list, and its load, source and meter slots take the first

channel too. That is consistent with popups built around one device per role, but it means a two-channel meter appears once rather than twice and its second channel is unreachable from that popup.

`getDevice()` does not check capability

It matches on the tag alone, so a handle it returns may not support the function the script then calls. The documentation's advice, to check that the function exists before calling it, is doing real work here.

`read...` and `get...` answer different questions

A feature's required-function list is specific about which one it wants. `readVoltage` needs a column in `#askValues` to point at, and if the column layout is mode-dependent, the index must be valid in every mode where the function is expected to work.

The cost of a `read...` call

A `read...` call asks for a sample of every loaded device before answering, suppressed only if one landed within the last 100 ms to 1.5 s. A script polling `readVoltage` faster than that window gets repeats of one sample rather than fresh readings, and a script polling it slower drives an acquisition cycle across the whole bench each time. On a bench where one slow instrument shares a gateway with the device being read, that slow instrument is in the cycle too.

6. What the documentation says, and where it stops

The mechanism is documented, but in three places and never as a general rule.

Page	What it contributes
Interface definitions	The clearest statement. <code>#interface</code> "is not mandatory ... but if it is missing the device may not be supported everywhere in TC"; <code>#interfaceType</code> "is not mandatory ... but is useful for classifying the driver, it can be added without any <code>#interface</code> definitions"; "Generally scripts and popups are supposed to check if a specific <code>#interface</code> exist before calling it". Plus the Primary / Secondary / Tertiary function sets per type — Primary being "needed, if the device must support all popups in TC".
ConfigDevice	Documents <code>#interfaceType</code> only as the way a script fetches a generic device, plus the aside "Generally the <code>#interfaceType</code> defines the <code>#interface</code> , only Virtual and PC are excepted from this rule" — which reads as a statement about convention, not about enforcement.
PopupBatteryTest	Shows one feature's pair concretely and correctly, as a flat per-role list: "For it to be used it must have these commands in the definition: <code>#interfaceType ps / #interface setCurrent / ...</code> " — a worked example rather than a stated rule.

Documented nowhere, as far as this note can establish:

- that the omission is silent;
- that a driver with no `#interfaceType` is excluded even by `\*` patterns;
- that the type requirement is evaluated before the function list, so a wrong tag makes the function list irrelevant;
- that some patterns exclude channel-suffixed tags, and which;
- that the Power supply test picks its source by first match in a fixed ACPS → PS → Power order and offers no choice between qualifying devices;

- that `getOn` is Primary but required by no feature, while `setRelays`, plural, is required by one.

The gap is not only visible from outside. Asked in March 2026 for a validator, the author noted that the manual carries only the three-group summary of `set...`, `get...` and `read...`, and added that "a list of device type and exactly how the minimum #interface is supposed to be defined would be a good addition" ([reply #6016](#)).

Two things in the documentation look like errors rather than gaps: the `thermometer` / `temperature` spellings noted above, and the [interface-definitions page](#) ending at "Secondary PWM" although its own table of contents promises AInput, AOutput, Calibrator and Notes sections after it.

None of this is a complaint. The behaviour is consistent and reasonable, and it is the kind of detail that is obvious to the person who wrote it and easy to miss from outside. The value in writing it down is that it becomes a checklist.

7. Checking a driver against this

Three things to try before concluding that TestController is ignoring a driver.

The [Test interface popup](#) is the only place in TestController that shows the declared types and the full interface map together, and it lets each function be called by hand, which is what the author recommends it for: "You can use the popup "Test interface for current selected device" to test that the functions are working correctly" ([reply #2446](#)). It also generates a `getDevice("...")` script preamble using the declared tag, which is a quick way to confirm the tag is what you think it is. It works on drivers with no type at all, so it is also the way to discover that the tag is what is missing.

Reading the tag back from a script narrows the problem to one half. If `getDevice("PS")` returns nothing while a supply is loaded, the tag is the problem rather than the function list.

Comparing against the table in section 3 one function at a time is the way through the other half. Because the function list is an AND with no reporting, the question to ask is which single name in that row is missing, rather than whether the driver supports the Battery test as a whole.

Appendix A — the matching rules as regular expressions

The tables above give each rule in words. Written as regular expressions, matched whole against one normalised tag, they are:

Power supply test	load	(load)(:\[0-9]+)?	
	source 1	(acps)(:\[0-9]+)?	
	source 2	(ps)(:\[0-9]+)?	
	source 3	(power)(:\[0-9]+)?	
	temperature	(thermometer)(:\[0-9]+)?	
Battery test	meters	(dmm)	
	load	(load)(:\[0-9]+)?	
	supply	(ps)(:\[0-9]+)?	
	relay	(relay)(:\[0-9]+)?	
	temperature	(thermometer)(:\[0-9]+)?	
MPPT	meters	(dmm)	
	load	(load)	
Remote readout	device	(readout)(:\[0-9]+)?	
Log Event	device	any tag	
Log Event Capture	device	any tag	
PSLoadReadout		(ps load charger)(:\[0-9]+)?	
MultiPSLoadReadout		(ps load)(:\[0-9]+)?	
SetPSVltCurrentWithOff		(ps acps)	
SetDualPSVltCurrentWithOff		(ps acps)	twice, one per dropdown
SetMultiLoadsWithOff		(load)	
OnOff, MultiOnOff		any tag	
SetDualParamsWithOff		any tag	
SetTripleParamsWithOff		any tag	
VoltCurrentPowerReadout		any tag	
EfficiencyReadout		any tag	three separate slots
DevicePopups		any tag	

Casing is shown lower-case throughout because tag normalisation settles it before the comparison. What you write in the driver does not have to match the case here, with one caveat below.

Each pattern is matched whole against one tag at a time, and the device passes on the first tag that matches. That is what makes the four rules without a `(:\[0-9]+)?` clause, the two DMM meter lists, MPPT and the two PS/Load setter panels, less consequential than they look. A device declaring `dmm dmm:2` still passes on `dmm`, and the clause's absence only means the suffixed tag would not have matched by itself. Whether the device's channels get offered is a separate question, answered by the "Ch" column in section 3 and not by the pattern. The only driver these four rules exclude is one that declares a suffixed tag with no bare tag beside it.

The caveat about casing: three of these patterns, MPPT's `Load`, `(PS|ACPS)` and `(Load)`, are matched case-sensitively, unlike the rest, which carry an explicit `(?i)`. They work because normalisation has already produced exactly `Load`, `PS` and `ACPS` by the time the comparison happens. Nothing a driver can write breaks them, but they depend on the normalisation step rather than on the pattern, which is relevant given that normalisation is the newest part of this machinery.

Confidence and provenance

Method	The published documentation, the author's public posts in the EEVblog thread, and observation of how TestController behaves while writing and debugging drivers for it. It carries no endorsement from the author.
Version	TestController 3.49. Every constant in section 3 and Appendix A belongs to that release.
Re-verify after	any TestController upgrade. Section 3 is a list of fixed values, and that is the kind of thing that changes quietly between releases.
Written	2026-08-20, revised 2026-08-21

Confidence is not uniform across this note. Three tiers:

Tier	What is in it	Standing
Quoted from a public source	The <code>set</code> / <code>get</code> / <code>read</code> distinction and the guidance to use <code>get...</code> for setpoints, the <code>:n</code> channel declaration, the Battery test's meter requirement, Steps not checking the device type, and the absence of any warning	The author's own words, in the documentation or in the thread, each quotation linked under Sources. They are statements about <code>TestController</code> , not about this note.
Version-sensitive	How the tag list is split and normalised (section 1)	Correct as observed on 3.49. A March 2026 exchange in the thread shows the same declaration behaving differently, so this table should not be assumed to hold on another release.
This note's own reading, unverified	Section 3's per-feature table, Appendix A's patterns, and the consequences drawn in sections 4 and 5	No public statement was found that sets out which popup or panel requires which function, so this is inference from observed behaviour. It is the largest table here and the most likely place for an error. Re-verify per release.

Where something is a deduction rather than something directly observed, the text says so.

Two claims in the first version of this note were wrong and are corrected above: that a meter declared `dmm dmm:2` was excluded from the meter lists, and that a `read...` call returned a stored value without querying anything. Anything in section 3 could be wrong the same way.

Sources

Documentation, on the author's site:

Page	Cited for
Interface definitions	<code>#interface</code> and <code>#interfaceType</code> being optional and independent; the Primary / Secondary / Tertiary function sets per type; <code>setRelays</code> / <code>getRelays</code> and <code>readInputs</code> ignoring the channel a channel-specific name would use
ConfigDevice	<code>#interfaceType</code> as the way a script fetches a generic device
PopupBatteryTest	the per-role list of required type and functions
Test interface popup	the popup that shows types and interface map together
Script commands	<code>getDevice()</code> , <code>#popupdevice</code> , <code>#SCRIPTINTERFACE</code>

Public posts by HKJ in EEVblog topic 234726, *Program that can log/control many multimeters and other devices*. Quoted briefly and linked rather than reproduced:

Post	Date	Quoted for
reply #1223	17 Oct 2020	"OSC and OSC:1 has the same meaning"
reply #2446	17 May 2022	what <code>#interface</code> is used for, and testing it from the Test interface popup
reply #3319	17 Jan 2024	<code>read...</code> taking <code>#askValues</code> column indices, one per channel

Post	Date	Quoted for
reply #5870	8 Mar 2026	<code>Relay Relay:2 Relay:3 Relay:4</code> declaring four independently addressed devices
reply #5924	13 Mar 2026	the comma in <code>#interfaceType DMM, BMM</code> , and the device not being recognised
reply #5959	14 Mar 2026	"the meter must be type DMM and support <code>readValue()</code> "
reply #6016	15 Mar 2026	the manual lacking a list of device type and minimum <code>#interface</code>
reply #6020	16 Mar 2026	why there is no warning for an incomplete definition
reply #7270	19 Aug 2026	<code>get...</code> rather than <code>read...</code> for setpoints; Steps not checking the device type; the <code>set...</code> name conventions making automatic use possible "in some popups"; "TC has no idea what the different number controls do"

Everything else here is observation of the program's behaviour while writing drivers for it, and is this note's own responsibility.

The author of TestController is the authority on all of this. Anything above that contradicts him is an error in this note.